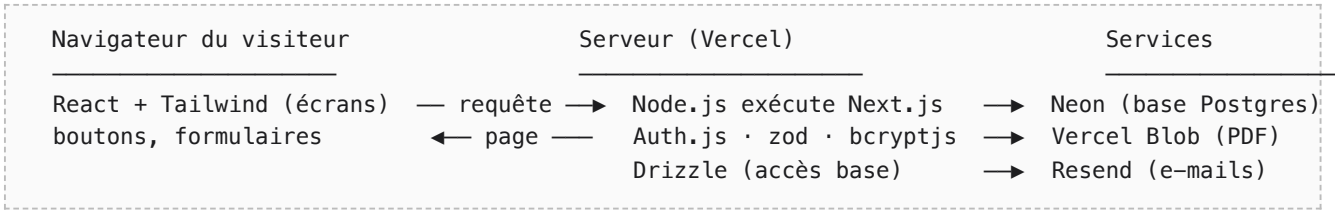
Infrastructure du projet

Agence-musiciens · langages, paquets et versions · relevé le 12 octobre 2026

> **À quoi sert cette fiche ?** Un site web n'est pas écrit « à partir de rien » : on assemble un langage, un moteur qui l'exécute, et des **paquets**, c'est-à-dire des morceaux de code écrits et maintenus par d'autres, qu'on installe plutôt que de tout réécrire. Cette fiche dit, pour chaque pièce, *ce que c'est*, *à quoi elle sert dans le projet* et *quelle version est installée*.
>
> **D'où viennent les versions ?** `package.json` donne la liste des paquets demandés ; `package-lock.json` enregistre la version exacte réellement installée. Les versions ci-dessous viennent de ce second fichier.



1. Le moteur : faire tourner le code

Node.js `24.21.0 (sur le Mac)`

Ce que c'est : le programme qui exécute du JavaScript *en dehors* du navigateur. Au départ, JavaScript ne fonctionnait que dans les pages web ; Node.js a sorti son moteur du navigateur et lui a donné accès aux fichiers et au réseau.

Dans le projet : c'est lui qui fait tourner le serveur : il reçoit les demandes, vérifie les droits, parle à la base de données. Sans lui, Next.js ne démarre pas.

Analogie : le piano. Sans instrument, la partition reste muette.

npm `11.19.0`

Ce que c'est : le gestionnaire de paquets livré avec Node.js.

Dans le projet : `npm install` télécharge tous les paquets listés ; `npm run dev` lance le site en local ; `npm test` lance les tests.

Analogie : le bibliothécaire qui va chercher les partitions dont tu as besoin, dans la bonne édition.

2. Les langages : dans quoi le code est écrit

TypeScript `5.9.3`

Ce que c'est : du JavaScript auquel on ajoute des **types** : on précise qu'une valeur est un texte, un nombre, une date… TypeScript vérifie *avant l'exécution* qu'on ne mélange pas tout, puis il est traduit en JavaScript ordinaire.

Dans le projet : tout le code (`.ts`). Exemple : si une fonction attend un identifiant d'engagement et qu'on lui passe une date, l'erreur est signalée dès l'écriture, pas devant l'agent.

Analogie : un correcteur qui relit la partition et signale une mesure à 5 temps dans un morceau à 4/4.

TSX

Ce que c'est : du TypeScript dans lequel on peut écrire des balises qui ressemblent au HTML (`<button>` , `<form>` …).

Dans le projet : les fichiers `.tsx` de `src/app/` et `src/components/` décrivent les écrans : à quoi ressemble la page et ce qu'elle affiche.

CSS (écrit via Tailwind)

Ce que c'est : le langage de la mise en forme : couleurs, tailles, espacements, mise en page.

Dans le projet : plutôt que d'écrire des fichiers CSS à part, on met de petites classes directement sur les éléments (`p-4` = marge intérieure, `text-sm` = petit texte). Fichier commun : `src/app/globals.css` .

SQL

Ce que c'est : le langage des bases de données relationnelles (créer des tables, lire, écrire).

Dans le projet : on ne l'écrit pas à la main. Drizzle le génère dans le dossier `drizzle/` à partir du schéma écrit en TypeScript : ce sont les **migrations**, les étapes successives de construction de la base.

3. Les paquets de l'application

Ces paquets sont utilisés par le site en fonctionnement (`dependencies`).

Next.js — `next` `16.4.0`

Ce que c'est : un **framework** : un cadre qui impose une organisation et fournit les mécanismes de base d'un site.

Dans le projet : chaque dossier de `src/app/` devient une adresse du site ; les **Server Actions** (`actions.ts`) exécutent sur le serveur ce que déclenche un formulaire.

Analogie : la forme sonate. Elle dit où placer chaque élément pour que l'ensemble tienne.

React — `react` , `react-dom` `19.3.0`

Ce que c'est : une bibliothèque pour construire une interface à partir de **composants** réutilisables (un bouton, une carte, un tableau). `react-dom` les affiche dans la page.

Dans le projet : tous les écrans sont des assemblages de composants React. Next.js est construit au-dessus de React.

Auth.js — `next-auth` `5.0.0-beta.32`

Ce que c'est : la bibliothèque d'authentification : savoir *qui* est connecté.

Dans le projet : gère la connexion par e-mail et mot de passe, puis range l'identité et le rôle dans un **cookie chiffré** avec `AUTH_SECRET` . Impossible à lire ou à modifier sans ce secret. Le contrôle des droits (« a-t-il le droit ? ») reste dans notre code.

Analogie : l'accueil de la salle, qui pose un bracelet infalsifiable ; chaque porte vérifie ensuite la couleur du bracelet.

bcryptjs `3.0.3`

Ce que c'est : un outil de **hachage** des mots de passe : il transforme un mot de passe en une empreinte à sens unique, impossible à retransformer en mot de passe.

Dans le projet : la base ne contient jamais les mots de passe, seulement leurs empreintes. À la connexion, bcrypt recalcule l'empreinte du mot de passe saisi et compare.

zod `4.6.5`

Ce que c'est : un outil de **validation** : on décrit la forme attendue d'une donnée (« un e-mail », « un cachet positif », « une date »), et zod refuse ce qui ne correspond pas.

Dans le projet : tout ce qui arrive d'un formulaire passe par zod côté serveur avant d'être enregistré. On ne fait jamais confiance au navigateur.

Analogie : le contrôle à l'entrée du conservatoire, qui vérifie que le dossier est complet avant de le transmettre au jury.

Drizzle — `drizzle-orm` `0.45.4`

Ce que c'est : un **ORM** : un traducteur entre le code TypeScript et la base de données.

Dans le projet : on écrit les tables et les requêtes en TypeScript ; Drizzle les traduit en SQL. Avantage : TypeScript vérifie aussi les requêtes.

Neon — `@neondatabase/serverless` `1.2.0`

Ce que c'est : le « câble » qui relie le serveur à la base Postgres hébergée chez Neon.

Dans le projet : Drizzle s'en sert pour envoyer ses requêtes à la base (utilisateurs, engagements…).

Vercel Blob — `@vercel/blob` `2.8.1`

Ce que c'est : un espace de stockage de fichiers.

Dans le projet : conserve les PDF joints aux engagements (contrats, programmes). La base ne stocke que la référence du fichier.

Resend — `resend` `6.32.1`

Ce que c'est : un service d'envoi d'e-mails, piloté depuis le code.

Dans le projet : envoie les invitations et les liens de réinitialisation du mot de passe.

Les briques d'interface

`shadcn` `4.21.4` — une collection de composants prêts à l'emploi (boutons, champs, tableaux), *copiés* dans `src/components/ui/` : on peut donc les modifier librement.

`@base-ui/react` `1.9.0` — les fondations de ces composants : clavier, lecteurs d'écran, accessibilité.

`class-variance-authority` `0.7.1` — décrit les déclinaisons d'un composant (bouton principal, secondaire, « danger »…).

`cn` `0.4.0` — assemble proprement les classes de style d'un composant.

`lucide-react` `1.55.0` — la bibliothèque d'icônes.

`tw-animate-css` `1.4.0` — de petites animations (apparition d'un menu…).

4. Les outils de développement

Utilisés pour écrire, vérifier et construire le code (`devDependencies`). Ils ne sont pas utilisés par le site une fois en ligne.

typescript `5.9.3` **et les** `@types/…`

Le vérificateur de types. Les paquets `@types/node` `24.19.2` , `@types/react` et `@types/react-dom` `19.3.0` lui décrivent les types de Node et de React, qui sont écrits en JavaScript.

tailwindcss et @tailwindcss/turbopack `4.3.3`

Lit les classes utilisées dans les composants et produit le fichier CSS final, en ne gardant que ce qui sert.

drizzle-kit `0.31.11`

Compare le schéma TypeScript à la base, génère les migrations SQL et les applique. Il est lancé automatiquement à chaque déploiement sur Vercel.

tsx `4.23.15`

Exécute directement un fichier TypeScript, sans étape de traduction. Sert à lancer le script du jeu de données de démonstration (`npm run donnees`).

vitest `5.0.3`

Lance les **tests automatiques** (`npm test`) : de petits programmes qui vérifient que le code fait ce qu'on attend, à chaque modification, sur le Mac et sur GitHub.

eslint `9.39.5` **et eslint-config-next** `16.4.0`

Relit le code et signale les erreurs probables et les mauvaises pratiques, avec les règles recommandées par Next.js.

5. À garder en tête

> **Auth.js est en version bêta** (5.0.0-beta.32) : une mise à jour peut changer son fonctionnement. On la vérifie avant de mettre à jour.
>
> **Version de Node.js** : 24.21.0 est celle du Mac. Celle de Vercel se règle dans les paramètres du projet ; elle n'a pas encore été vérifiée. Les deux doivent correspondre.
>
> **Le symbole** `^` dans `package.json` signifie « cette version ou une plus récente de la même série ». C'est pourquoi on relève la version exacte dans `package-lock.json` .

Mettre à jour les versions de cette fiche : depuis le dossier du projet, la commande suivante affiche la version installée de chaque paquet déclaré.

```
node -e 'const l=require("./package-lock.json"),p=require("./package.json");
for (const n of Object.keys({...p.dependencies,...p.devDependencies})) console.log(n,
```

```
l.packages["node_modules/"+n].version)'
```